

Module 5 Lab:

Capture & Replay Testing with Selenium

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

PART 1: Selenium Setup & Website Selection

Selenium WebDriver and pytest were installed and verified on macOS using Python 3.14. Firefox was selected as the automation browser due to ChromeDriver extension deprecation. A project directory (SeleniumLab5) was created and the four baseline scripts from the lab instructions were executed successfully.

Two websites were selected for original testing: **DuckDuckGo** and **USGS**. Both provide stable, publicly accessible search interfaces without login walls or CAPTCHA. Several other candidates were evaluated and rejected due to automation barriers (CAPTCHA, dynamic modals, or unstable DOMs), which informed the final selection.

PART 2: Testing Scripts

Four original scripts were created using Selenium IDE and exported to Python:

- DuckDuckGo – normal search
- DuckDuckGo – resistance test
- USGS – normal navigation/search
- USGS – resistance test

Each script follows the same workflow: open site, locate search element, enter input, submit, wait for results, and close the browser. Minor manual edits were required after export due to Selenium IDE inserting unnecessary frame-switching steps and inconsistently exporting pause commands. These issues were corrected in the Python scripts.

PART 3: Resistance Training

Resistance testing evaluated how each site handled invalid or unexpected input. DuckDuckGo returned a stable no-results page for nonsense input without errors. USGS also handled invalid input gracefully, returning a no-results state or stable navigation response.

Additional resistance findings emerged during site selection. Bing blocked automation with a human-verification CAPTCHA. Goodreads introduced a login modal during playback that did not appear during recording, causing synchronization failures. A personal Blogger site rendered its search box dynamically, causing Selenium to time out despite the element being visible. These cases illustrate three real-world resistance categories: anti-bot protection, modal interruption, and dynamic element rendering.

PART 4: Test Case Reuse

All four original test scripts share a common reusable workflow pattern, even though the USGS normal test required a navigation-based flow rather than a search. The reusable structure is:

1. Launch WebDriver
2. Navigate to target URL
3. Locate an interactive element (search box or menu link)
4. Enter input or click navigation elements
5. Submit the action (ENTER or click)
6. Wait for the resulting page to load
7. Close the browser

This pattern supports reuse because the underlying logic remains the same across scenarios: identify an element, perform an interaction, and validate that the application responds without error.

For **DuckDuckGo**, both the normal and resistance tests reuse the same search-based workflow, differing only in the input string. For **USGS**, the normal test reuses the same structural

pattern but substitutes menu navigation for search input due to the site's search replay instability. The resistance test mirrors this navigation flow with invalid input to confirm stable behavior.

In a production environment, this shared structure would be refactored into a single parameterized function or pytest fixture accepting URL, locator strategy, locator value, and input. The `setup_method` / `teardown_method` pattern already demonstrates reuse by centralizing browser lifecycle management and eliminating repeated boilerplate.

PART 5: Control Flows

The control flow for all tests is linear with one conditional branch: whether the search element is found. If located, the test proceeds; if not, Selenium raises an exception and the test fails. This behavior was observed directly when testing the Blogger site, which produced a `TimeoutException` after 30 seconds. Flow diagrams are included in Appendix 4.

PART 6: Testing Results

All eight test cases (four baseline, four original) executed successfully. DuckDuckGo and USGS demonstrated stable behavior under both valid and invalid input. No crashes, HTTP errors, or unstable states were observed. Execution time ranged from 15–25 seconds including pauses. Full results and screenshots are included in Appendices 1 and 2.

PART 7: Critical Evaluation of Selenium

Selenium provides strong cross-browser automation capabilities, broad language support, and a mature WebDriver API, making it well-suited for regression testing of internally controlled applications with stable DOM structures. Its integration with Python and pytest enables maintainable test suites and repeatable workflows.

However, this lab highlighted several practical limitations. Modern public websites frequently deploy anti-bot protections, dynamic rendering, or modal interruptions that break

capture-and-replay workflows. Bing blocked automation with a CAPTCHA, Goodreads introduced a login modal during playback, and a Blogger site rendered its search box dynamically enough that Selenium could not locate it despite being visually present. These failures illustrate Selenium's sensitivity to timing, DOM volatility, and client-side rendering.

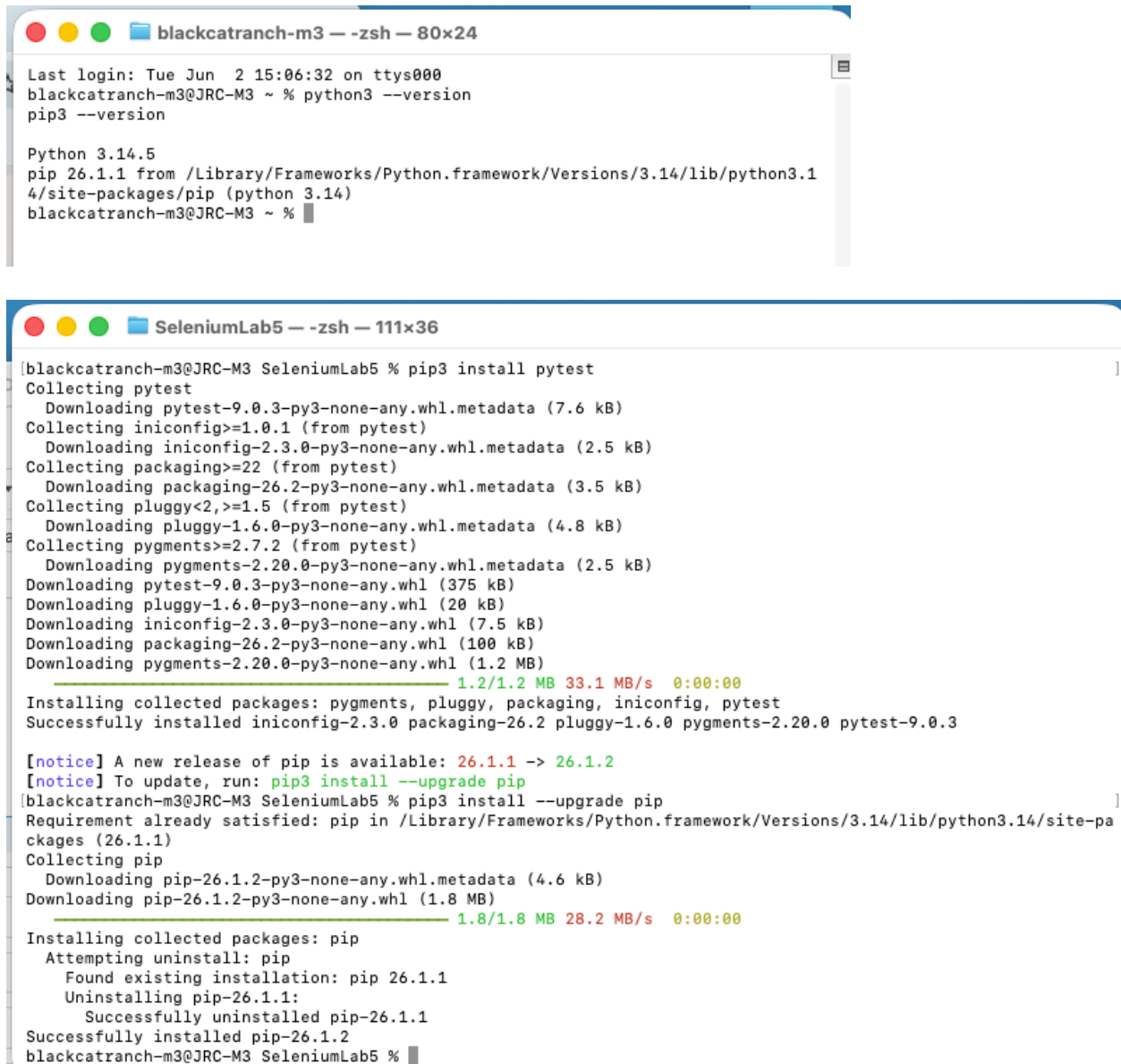
Tooling stability is also a concern. The Selenium IDE extension is no longer supported in Chrome, requiring the use of Firefox for recording. This browser-ecosystem fragility reinforces that Selenium's reliability depends not only on the application under test but also on shifting browser policies and extension support.

Recommendation: Selenium is best suited for long-running regression suites on applications where the development team controls the UI and can ensure stable locators. It is less effective for consumer-facing sites with CAPTCHA, login walls, or heavy dynamic content, or for one-off tests where manual validation is faster. For highly dynamic modern sites, tools like Playwright or Cypress may offer more resilient auto-waiting and synchronization behavior.

Selenium's long-term value also depends on treating automated tests as a maintained library rather than one-off scripts. In controlled environments, such as internal applications or accessibility/WCAG regression suites, teams can design stable locators, enforce consistent DOM patterns, and update tests alongside feature changes. This allows Selenium scripts to be reused across many cycles with predictable results. In contrast, public commercial sites change frequently, introduce dynamic content, and deploy anti-bot protections, making it difficult to sustain a reliable automation suite without constant rework.

APPENDIX 1: Set-up & Test Scripts

Figure 1a and 1b: Terminal showing installation of Python and pip



```
blackcatranch-m3 — -zsh — 80x24

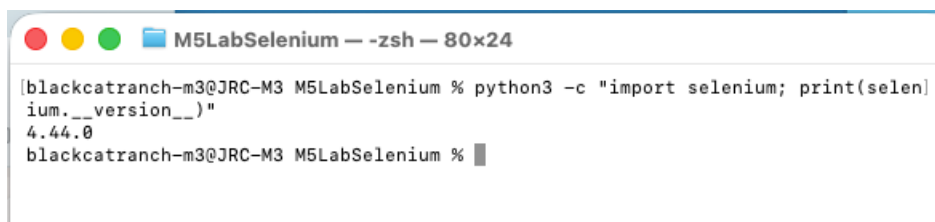
Last login: Tue Jun  2 15:06:32 on ttys000
blackcatranch-m3@JRC-M3 ~ % python3 --version
python3.14.5
blackcatranch-m3@JRC-M3 ~ % pip3 --version
pip 26.1.1 from /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages/pip (python 3.14)
blackcatranch-m3@JRC-M3 ~ %
```

```
SeleniumLab5 — -zsh — 111x36

blackcatranch-m3@JRC-M3 SeleniumLab5 % pip3 install pytest
Collecting pytest
  Downloading pytest-9.0.3-py3-none-any.whl.metadata (7.6 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.0.3-py3-none-any.whl (375 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
1.2/1.2 MB 33.1 MB/s 0:00:00
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
Successfully installed iniconfig-2.3.0 packaging-26.2 pluggy-1.6.0 pygments-2.20.0 pytest-9.0.3

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip3 install --upgrade pip
blackcatranch-m3@JRC-M3 SeleniumLab5 % pip3 install --upgrade pip
Requirement already satisfied: pip in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (26.1.1)
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
1.8/1.8 MB 28.2 MB/s 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 26.1.1
    Uninstalling pip-26.1.1:
      Successfully uninstalled pip-26.1.1
  Successfully installed pip-26.1.2
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

Figure 2: Verification of Selenium installation in Terminal



```
M5LabSelenium — -zsh — 80x24

blackcatranch-m3@JRC-M3 M5LabSelenium % python3 -c "import selenium; print(selenium.__version__)"
4.44.0
blackcatranch-m3@JRC-M3 M5LabSelenium %
```

Figure 3: Firefox Browser with Selenium IDE installed & pop-up for browser extension welcome screen

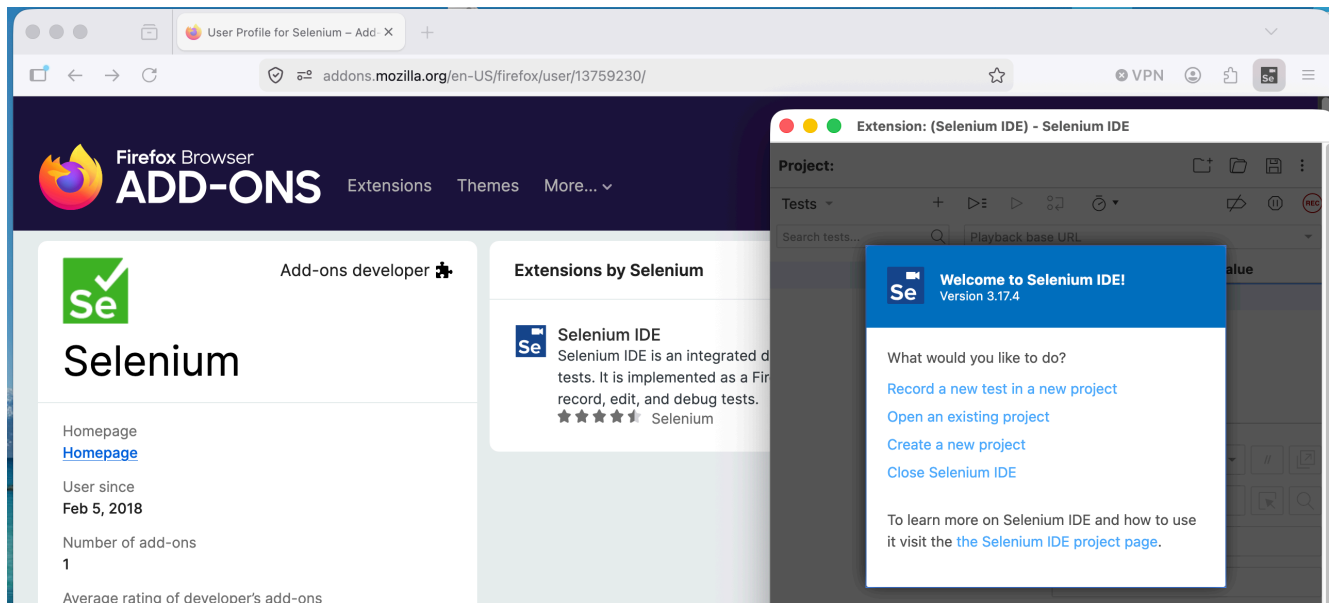


Figure 4: Terminal showing SeleniumLab5 folder with files for scripts

```
SeleniumLab5 — -zsh — 80x24

blackcatranch-m3@JRC-M3 SeleniumLab5 % touch test.py wikipedia_resistance.py python_search.py python_resistance.py

blackcatranch-m3@JRC-M3 SeleniumLab5 % ls -la
total 0
drwxr-xr-x  6 blackcatranch-m3  staff  192 Jun  3 22:05 .
drwxr-xr-x@ 9 blackcatranch-m3  staff  288 Jun  3 22:03 ..
-rw-r--r--  1 blackcatranch-m3  staff   0 Jun  3 22:05 python_resistance.py
-rw-r--r--  1 blackcatranch-m3  staff   0 Jun  3 22:05 python_search.py
-rw-r--r--  1 blackcatranch-m3  staff   0 Jun  3 22:05 test.py
-rw-r--r--  1 blackcatranch-m3  staff   0 Jun  3 22:05 wikipedia_resistance.py
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

Figure 5: Terminal showing test.py script from lab instructions

```
SeleniumLab5 — nano test.py — 79x28

UW PICO 5.09 File: test.py Modified

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://www.wikipedia.org")

search_box = driver.find_element(By.NAME, "search")

search_box.send_keys("Software Testing")

search_box.submit()

time.sleep(5)

print("Page Title:", driver.title)

input("Press Enter to close browser...")

driver.quit()

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell
```

Figure 6: Terminal showing Wikipedia_resistance.py file creation



The terminal window shows the nano editor interface for creating the file wikipedia_resistance.py. The title bar indicates the window is titled 'SeleniumLab5 — nano wikipedia_resistance.py — 79x28'. The status bar shows 'UW PICO 5.09' and 'File: wikipedia_resistance.py'. The code being written is as follows:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://www.wikipedia.org")

search_box = driver.find_element(By.NAME, "search")

search_box.send_keys("@@@@@@@@")

search_box.submit()

time.sleep(5)

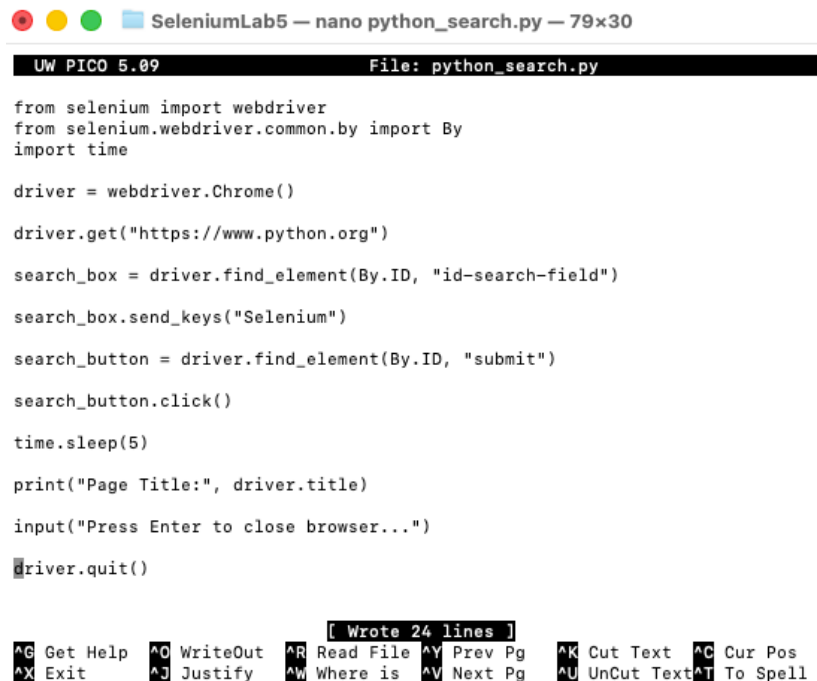
print("Page Title:", driver.title)

input("Press Enter to close browser...")

driver.quit()
```

At the bottom of the terminal, a status bar indicates '[Wrote 22 lines]' and provides keyboard shortcuts for various editor functions.

Figure 7: Terminal showing python_search.py file creation



The terminal window shows the nano editor interface for creating the file python_search.py. The title bar indicates the window is titled 'SeleniumLab5 — nano python_search.py — 79x30'. The status bar shows 'UW PICO 5.09' and 'File: python_search.py'. The code being written is as follows:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://www.python.org")

search_box = driver.find_element(By.ID, "id-search-field")

search_box.send_keys("Selenium")

search_button = driver.find_element(By.ID, "submit")

search_button.click()

time.sleep(5)

print("Page Title:", driver.title)

input("Press Enter to close browser...")

driver.quit()
```

At the bottom of the terminal, a status bar indicates '[Wrote 24 lines]' and provides keyboard shortcuts for various editor functions.

Figure 8: Terminal showing python_resistance.py creation

```
UW PICO 5.09 File: python_resistance.py

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://www.python.org")

search_box = driver.find_element(By.ID, "id-search-field")

search_box.send_keys("@@@@@@")

search_button = driver.find_element(By.ID, "submit")

search_button.click()

time.sleep(5)

print("Page Title:", driver.title)

input("Press Enter to close browser...")

driver.quit()

[ Wrote 24 lines ]
```

Figure 9. Wikipedia browser window controlled by test.py

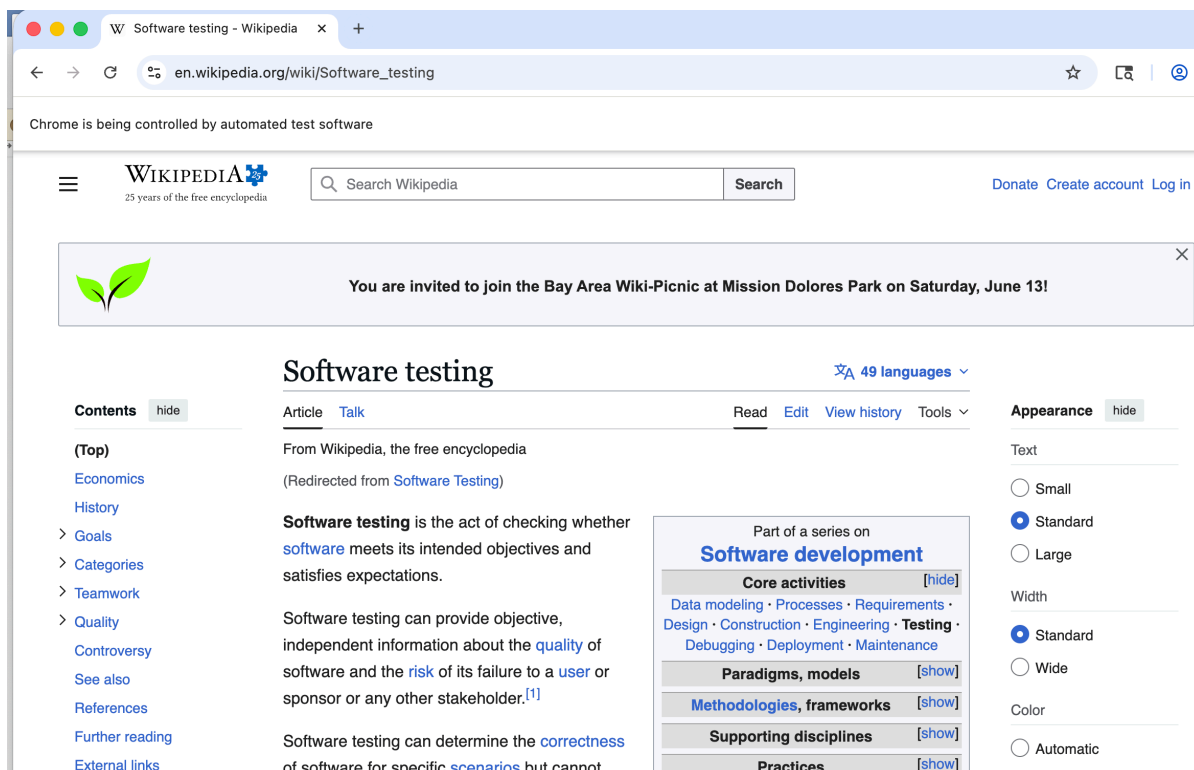


Figure 10. Wikipedia browser window controlled by wikipedia_resistance.py

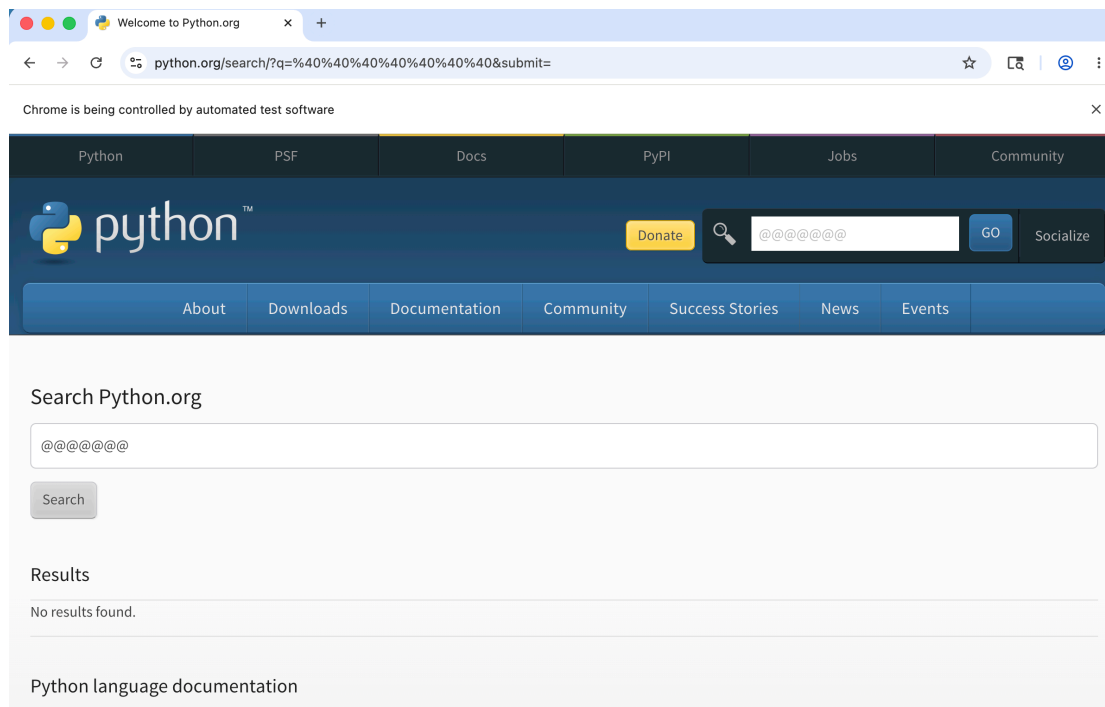


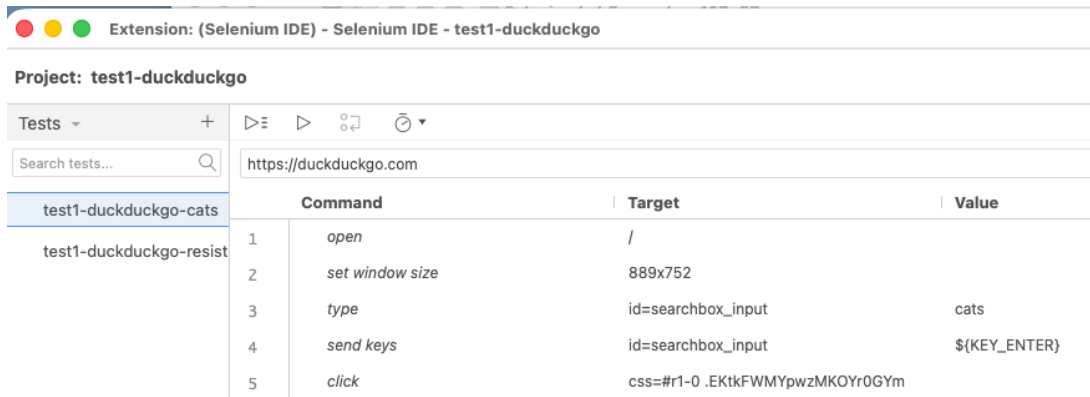
Figure 13. Terminal showing results of running all four scripts from lab instructions

```
SeleniumLab5 — -zsh — 79x20

[blackcatranch-m3@JRC-M3 SeleniumLab5 % nano python_resistance.py
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python3 test.py
Page Title: Software testing - Wikipedia
Press Enter to close browser...
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python wikipedia_resistance.py
zsh: command not found: python
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python3 wikipedia_resistance.py
Page Title: ##### - Search results - Wikipedia
Press Enter to close browser...
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python3 test.py
Page Title: Software testing - Wikipedia
Press Enter to close browser...
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python3 python_search.py
Page Title: Welcome to Python.org
Press Enter to close browser...
[blackcatranch-m3@JRC-M3 SeleniumLab5 % python3 python_resistance.py
Page Title: Welcome to Python.org
Press Enter to close browser...
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

APPENDIX 2: Record New Tests & Confirm Playback

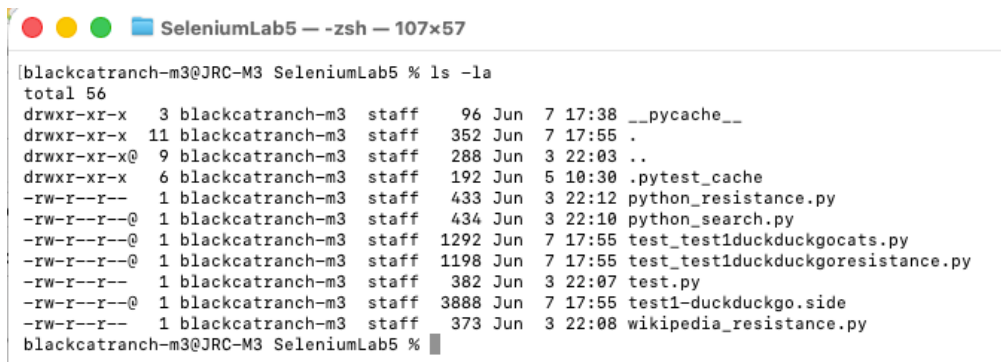
Figure 14. Selenium IDE Capture of New Test 1 – test1-duckduckgo



The screenshot shows the Selenium IDE interface. At the top, it says 'Extension: (Selenium IDE) - Selenium IDE - test1-duckduckgo'. Below that, the project name is 'test1-duckduckgo'. The 'Tests' tab is selected, and a search bar contains 'https://duckduckgo.com'. A table lists the commands for the test:

	Command	Target	Value
1	open	/	
2	set window size	889x752	
3	type	id=searchbox_input	cats
4	send keys	id=searchbox_input	\$(KEY_ENTER)
5	click	css=#r1-0 .EKtkFWMYpwzMKOYr0GYm	

Figure 15. Folder Listing Showing New Test Recording Projects and Exported .py Scripts



The screenshot shows a terminal window titled 'SeleniumLab5 -- zsh -- 107x57'. The user has run the command 'ls -la' in the directory '/blackcatranch-m3@JRC-M3 SeleniumLab5'. The output shows a list of files and directories:

```
total 56
drwxr-xr-x  3 blackcatranch-m3  staff   96 Jun  7 17:38 __pycache__
drwxr-xr-x 11 blackcatranch-m3  staff  352 Jun  7 17:55 .
drwxr-xr-x@ 9 blackcatranch-m3  staff  288 Jun  3 22:03 ..
drwxr-xr-x  6 blackcatranch-m3  staff  192 Jun  5 10:30 .pytest_cache
-rw-r--r--  1 blackcatranch-m3  staff  433 Jun  3 22:12 python_resistance.py
-rw-r--r--  1 blackcatranch-m3  staff  434 Jun  3 22:10 python_search.py
-rw-r--r--  1 blackcatranch-m3  staff 1292 Jun  7 17:55 test_testiduckduckgocats.py
-rw-r--r--  1 blackcatranch-m3  staff 1198 Jun  7 17:55 test_testiduckduckgoresistance.py
-rw-r--r--  1 blackcatranch-m3  staff  382 Jun  3 22:07 test.py
-rw-r--r--  1 blackcatranch-m3  staff 3888 Jun  7 17:55 test1-duckduckgo.side
-rw-r--r--  1 blackcatranch-m3  staff  373 Jun  3 22:08 wikipedia_resistance.py
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

Figure 16. Playback of test1-duckduckgo – Cats search

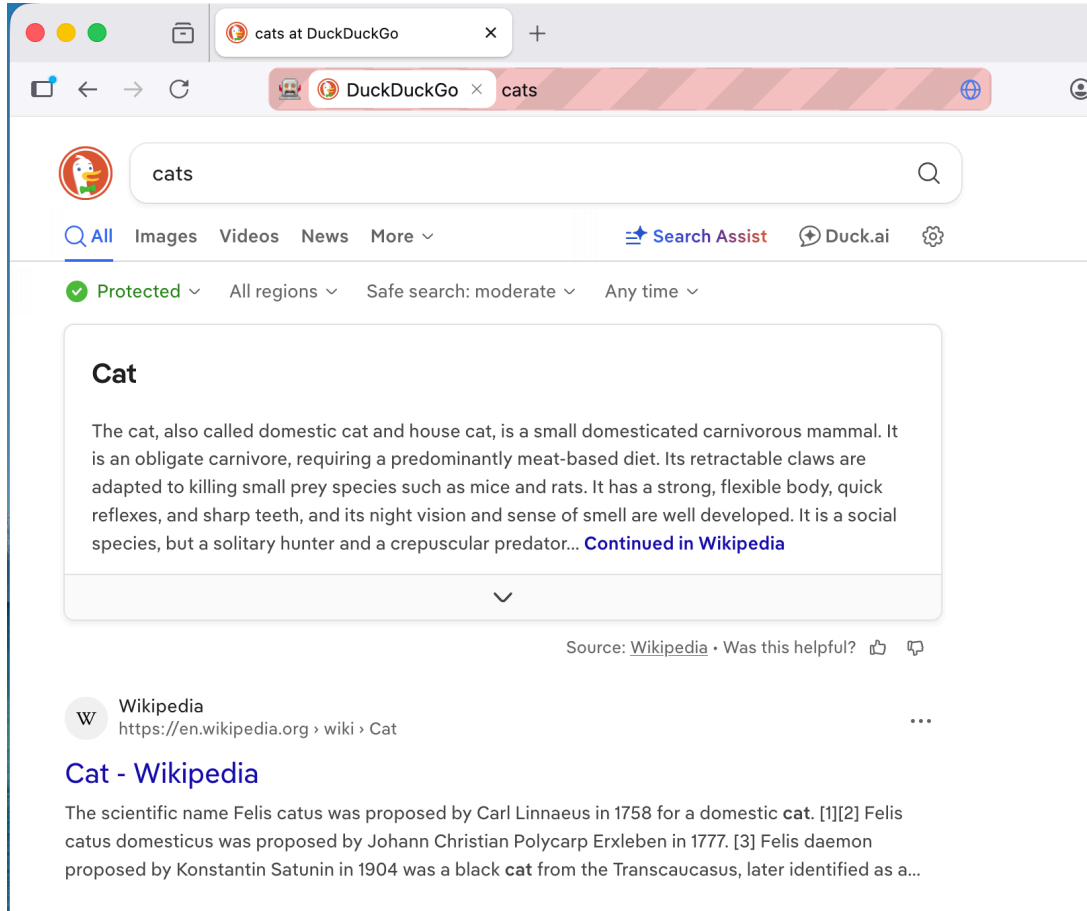


Figure 17. Terminal Results from Playback of test1-duckduckgo – Cats search

```
[blackcatranch-m3@JRC-M3 SeleniumLab5 % pytest test_test1duckduckgocats.py
===== test session starts =====
platform darwin -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/blackcatranch-m3/Desktop/CSYE6255/M05-CSYE6255/M5LabSelenium/SeleniumLab5
collected 1 item

test_test1duckduckgocats.py .                                     [100%]

===== 1 passed in 32.59s =====
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

Figure 18. Playback of test1-duckduckgo – Resistance test

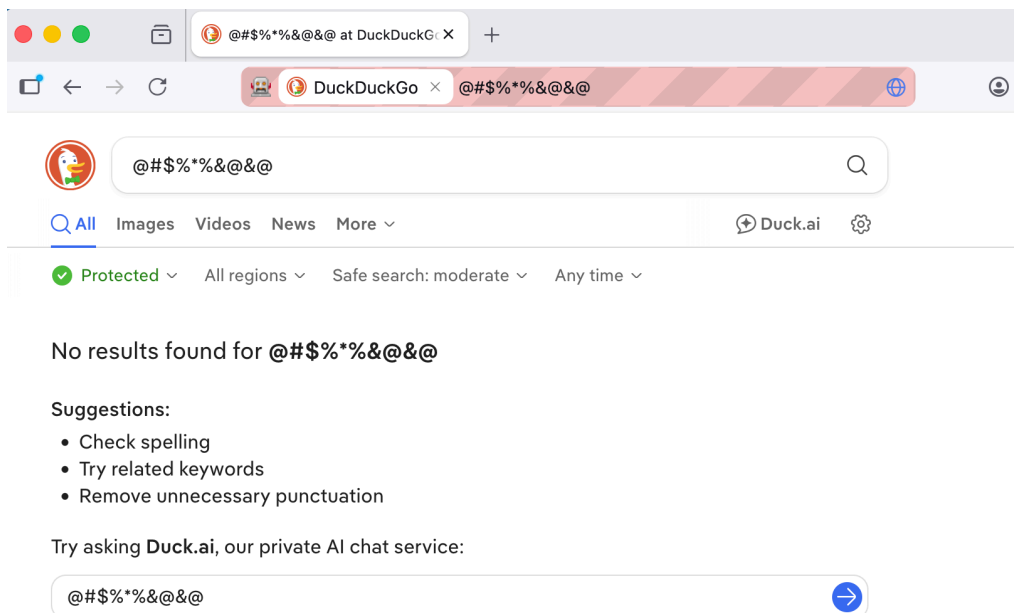


Figure 19. Terminal Results from Playback of test1-duckduckgo – Resistance test

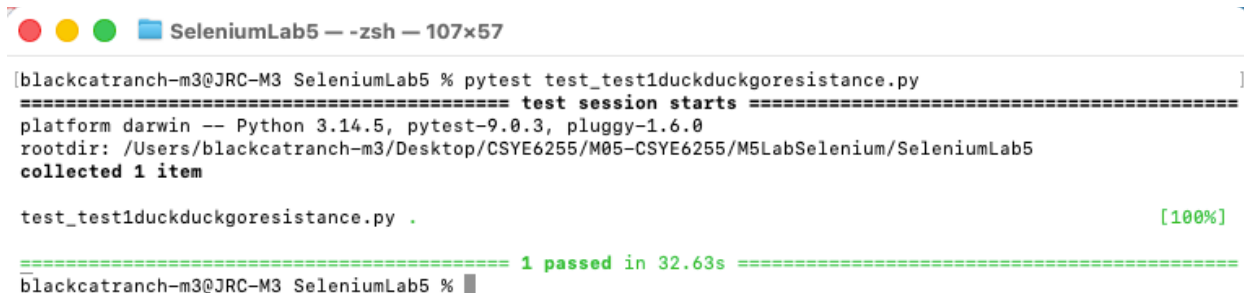


Figure 20. Selenium IDE & Firefox Capture of New Test 2

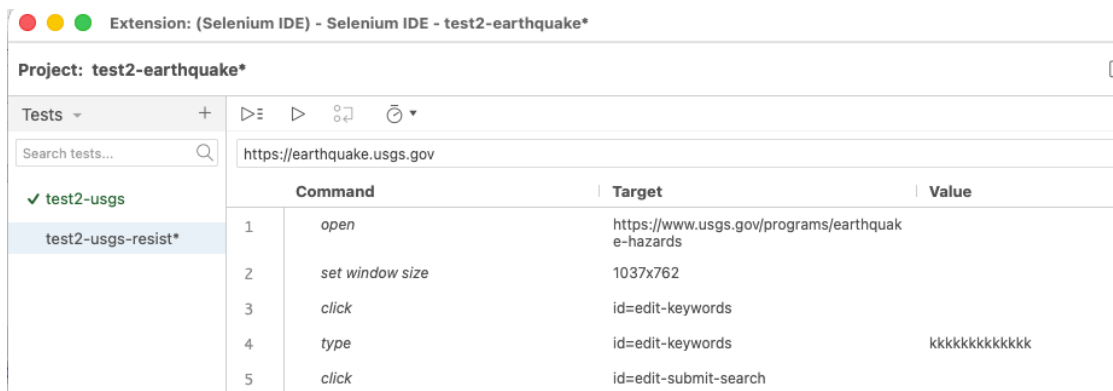


Figure 21. Playback of New Test 2

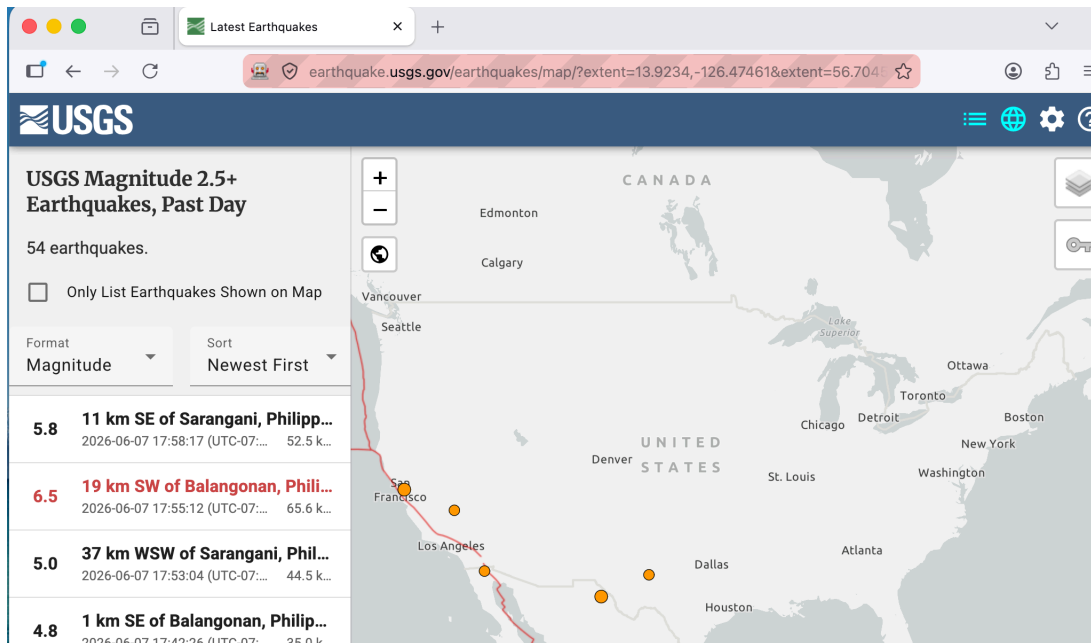
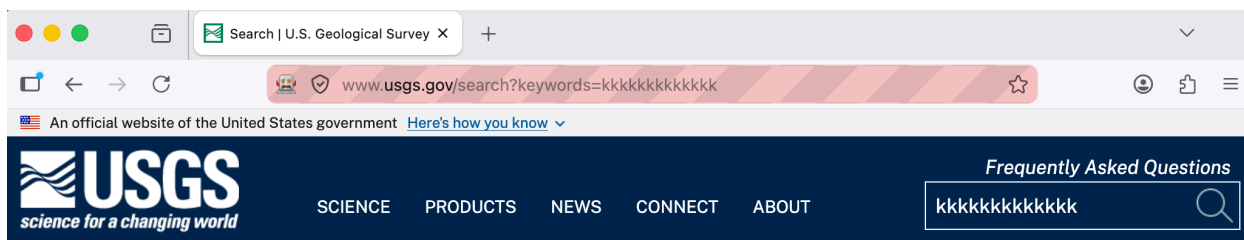


Figure 22. Terminal Results of New Test 2

```
[blackcatranch-m3@JRC-M3 SeleniumLab5 % pytest test_test2usgs.py  
===== test session starts =====  
platform darwin -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0  
rootdir: /Users/blackcatranch-m3/Desktop/CSYE6255/M05-CSYE6255/M5LabSelenium/SeleniumLab5  
collected 1 item  
  
test_test2usgs.py . [100%]  
  
===== 1 passed in 34.09s =====  
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

Figure 23. Playback of New Test 2 Resistance



Explore Search: kkkkkkkkkkkkkk

No results found.

Figure 22. Terminal Results of New Test 2 Resistance

```
blackcatranch-m3@JRC-M3 SeleniumLab5 % pytest test_test2usgsresist.py
===== test session starts =====
platform darwin -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/blackcatranch-m3/Desktop/CSYE6255/M05-CSYE6255/M5LabSelenium/SeleniumLab5
collected 1 item

test_test2usgsresist.py . [100%]

===== 1 passed in 33.21s =====
blackcatranch-m3@JRC-M3 SeleniumLab5 %
```

APPENDIX 3: Exported .py scripts from Selenium IDE tests

Figure 23. test_test1duckduckgocats.py

UW PICO 5.09

File: test_test1duckduckgocats.py

```
# Generated by Selenium IDE
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestTest1duckduckgocats():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test1duckduckgocats(self):
        # Test name: test1-duckduckgo-cats
        # Step # | name | target | value
        # 1 | open | / |
        self.driver.get("https://duckduckgo.com/")
        # 2 | setWindowSize | 889x752 |
        self.driver.set_window_size(889, 752)
        # 3 | type | id=searchbox_input | cats
        self.driver.find_element(By.ID, "searchbox_input").send_keys("cats")
        # 4 | sendKeys | id=searchbox_input | ${KEY_ENTER}
        self.driver.find_element(By.ID, "searchbox_input").send_keys(Keys.ENTER)
        # 5 | pause | 30000 |
        time.sleep(30)
```

Figure 24. test_test1duckduckgoresistance.py

```
UW PICO 5.09 File: test_test1duckduckgoresistance.py

# Generated by Selenium IDE
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestTest1duckduckgoresistance():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test1duckduckgoresistance(self):
        # Test name: test1-duckduckgo-resistance
        # Step # | name | target | value
        # 1 | open | / |
        self.driver.get("https://duckduckgo.com/")
        # 2 | setWindowSize | 889x752 |
        self.driver.set_window_size(889, 752)
        # 3 | type | id=searchbox_input | @#$%*%&@&@
        self.driver.find_element(By.ID, "searchbox_input").send_keys("@#$%*%&@&@")
        # 4 | sendKeys | id=searchbox_input | ${KEY_ENTER}
        self.driver.find_element(By.ID, "searchbox_input").send_keys(Keys.ENTER)
        # 5 | pause | 30000 |
        time.sleep(30)
```

Figure 25. test_test2usgs.py

```
UW PICO 5.09 File: test_test2usgs.py

# Generated by Selenium IDE
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestTest2usgs():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test2usgs(self):
        # Test name: test2-usgs
        # Step # | name | target | value
        # 1 | open | https://www.usgs.gov/programs/earthquake-hazards |
        self.driver.get("https://www.usgs.gov/programs/earthquake-hazards")
        # 2 | setWindowSize | 1037x761 |
        self.driver.set_window_size(1037, 761)
        # 3 | mouseOver | linkText=Significant Earthquakes Archive |
        element = self.driver.find_element(By.LINK_TEXT, "Significant Earthquakes Archive")
        actions = ActionChains(self.driver)
        actions.move_to_element(element).perform()
        # 4 | click | linkText=View Earthquakes |
        self.driver.find_element(By.LINK_TEXT, "View Earthquakes").click()
        # 5 | pause | 30000 |
        time.sleep(30)
```

Figure 26. test_test2usgsresist.py

```
UW PICO 5.09 File: test_test2usgsresist.py

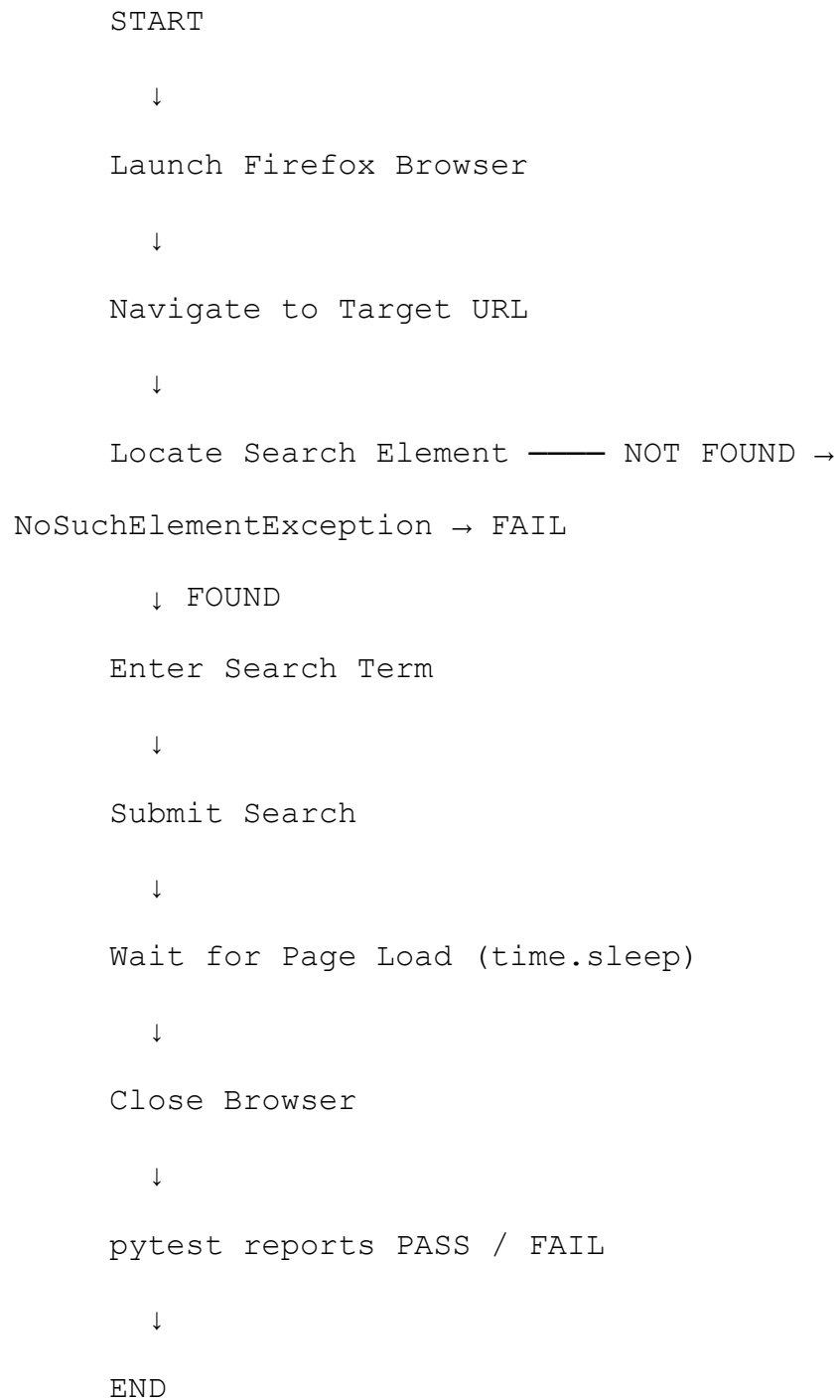
# Generated by Selenium IDE
import pytest
import time
import json
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.desired_capabilities import DesiredCapabilities

class TestTest2usgsresist():
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.vars = {}

    def teardown_method(self, method):
        self.driver.quit()

    def test_test2usgsresist(self):
        # Test name: test2-usgs-resist
        # Step # | name | target | value
        # 1 | open | https://www.usgs.gov/programs/earthquake-hazards |
        self.driver.get("https://www.usgs.gov/programs/earthquake-hazards")
        # 2 | setWindowSize | 1037x762 |
        self.driver.set_window_size(1037, 762)
        # 3 | click | id=edit-keywords |
        self.driver.find_element(By.ID, "edit-keywords").click()
        # 4 | type | id=edit-keywords | kkkkkkkkkkkkkk
        self.driver.find_element(By.ID, "edit-keywords").send_keys("kkkkkkkkkkkkkk")
        # 5 | click | id=edit-submit-search |
        self.driver.find_element(By.ID, "edit-submit-search").click()
        # 6 | pause | 30000 |
        time.sleep(30)
```

APPENDIX 4. Control Flows



Methodology and Use of AI Tools

Claude was used to troubleshoot script configuration. The analysis, comparisons, and written report reflect the author's own observations and conclusions.

Reference to Claude LLM conversation: <https://claude.ai/share/8bce9442-b71c-499d-9a05-9879f5709188>